

EXERCISE 2

POOSA VISHNUVARDHAN

90119953

LEVEL 1

Q1.

Data Types and Reasons

1. A customer's PAN number, always exactly 10 characters.
Data Type: CHAR(10)
Reason: PAN is a fixed-length alphanumeric value, so CHAR(10) stores exactly 10 characters.
2. The interest amount credited on a fixed deposit at maturity.
Data Type: DECIMAL(15,2)
Reason: Monetary amounts require exact decimal precision to avoid rounding errors.
3. The number of days a deposit runs, maximum 3,650.
Data Type: SMALLINT
Reason: The value is a whole number with a relatively small maximum value of 3,650.
4. The exact moment a UPI payment was authorised.
Data Type: TIMESTAMP
Reason: TIMESTAMP stores both the date and exact time, including fractional seconds if required.
5. The date a loan instalment falls due.
Data Type: DATE
Reason: Only the calendar date is required, not the exact time of day.
6. A free-text branch note about a customer complaint.
Data Type: VARCHAR(1000)
Reason: VARCHAR allows variable-length text and is suitable for storing customer complaint notes.
7. A running counter of every transaction the bank has ever posted.
Data Type: BIGINT
Reason: A transaction counter can grow to a very large whole number over time.
8. Whether a deposit auto-renews.
Data Type: BOOLEAN
Reason: The column has only two logical values: TRUE or FALSE.

Q2.

```
SELECT CAST(0.1 AS FLOAT) + CAST(0.2 AS FLOAT) AS float_result;
```

```
SELECT CAST(0.1 AS DECIMAL(10,2)) + CAST(0.2 AS DECIMAL(10,2)) AS decimal_result;
```

Business Impact: Using FLOAT for monetary values can introduce rounding errors, which may cause incorrect interest calculations, account balances, and financial records.

Q3. Constraints

Constraint	What it guarantees	One ICICI Bank example
NOT NULL	Guarantees that a column cannot contain a NULL value.	customer_name in a customer table must always be provided.
UNIQUE	Guarantees that all non-NULL values in a column are unique.	PAN number should be unique for each customer.
PRIMARY KEY	Uniquely identifies each row in a table and cannot be NULL.	customer_id uniquely identifies every customer.
FOREIGN KEY	Guarantees that a value refers to an existing row in another table.	customer_id in an account table references customer_id in the customer table.
CHECK	Guarantees that values satisfy a specified condition.	balance >= 0 ensures that an account balance cannot be negative.
DEFAULT	Automatically provides a value when no value is supplied during insertion.	auto_renew can default to FALSE when a deposit is created.

Q4. PRIMARY KEY vs UNIQUE Constraint

Two differences between PRIMARY KEY and UNIQUE constraint are:

1. A PRIMARY KEY uniquely identifies each row and does not allow NULL values, whereas a UNIQUE constraint ensures uniqueness but generally allows NULL values depending on the database system.
2. A table can have only one PRIMARY KEY, whereas a table can have multiple UNIQUE constraints.

Why deposit_id was chosen as the PRIMARY KEY rather than receipt_number:

deposit_id is a stable, system-generated identifier that uniquely identifies each deposit record and is intended for relationships with other tables. receipt_number is a business/reference number and may be subject to changes in business processes or formatting, so it is better enforced as UNIQUE rather than used as the primary key.

Q5. DELETE vs TRUNCATE vs DROP

Dimension	DELETE	TRUNCATE	DROP
What is removed	Removes selected rows from a table; without WHERE, all rows are removed.	Removes all rows from the table but keeps the table structure.	Removes the entire table, including its data and structure.

Dimension	DELETE	TRUNCATE	DROP
WHERE allowed	Yes	No	No
Can it be rolled back?	Yes, before COMMIT, in databases such as Oracle.	Yes, in Oracle before COMMIT; behavior differs across database systems.	Yes, in Oracle before COMMIT; behavior differs across database systems.
Do triggers fire?	DELETE triggers can fire.	DELETE triggers do not fire; a database-specific TRUNCATE/DDL trigger may apply.	DELETE triggers do not fire; DDL triggers may fire.
Speed on a large table	Generally slower because individual rows are processed and logged.	Generally faster because it removes all rows as a bulk operation.	Generally fastest for removing the table itself because the table definition is also removed.

Important: TRUNCATE and DROP should be used carefully in banking systems because they are powerful operations and can cause significant data loss.

Q6. Seven-Year Transaction Retention

What is wrong with the developer's job:

```
DELETE FROM account_txn
```

```
WHERE txn_date < CURRENT_DATE - 90;
```

The job deletes transactions older than 90 days, which directly conflicts with the requirement to retain transaction history for seven years. It could permanently remove records that the bank is required to preserve for regulatory, audit, dispute, and investigation purposes.

What the bank should do instead:

The bank should retain transaction records for at least seven years according to the stated regulatory requirement. A retention policy should be implemented using an approved archival process, where older transactions are moved to secure archival storage rather than deleted after 90 days.

For example:

```
-- Archive transactions older than the approved retention period
```

```
-- before considering deletion from operational storage.
```

The actual deletion should only occur after the legally approved retention period has expired and after applicable legal holds, audits, disputes, and compliance requirements have been checked.

Q7. NULL Demonstration

Assuming Oracle SQL:

1. Query: `SELECT NULL = NULL FROM dual;`

Output: NULL

Reason: NULL represents an unknown value. Comparing NULL with NULL does not produce TRUE; it produces UNKNOWN, which is displayed as NULL.

2. Query: SELECT NULL <> NULL FROM dual;

Output:NULL

Reason: The result is UNKNOWN because NULL cannot be compared using = or <>.

3. Query: SELECT 5000 + NULL FROM dual;

Output:NULL

Reason: Any arithmetic operation involving NULL results in NULL.

4. Query: SELECT COALESCE(NULL, NULL, 'FALLBACK') FROM dual;

Output:FALLBACK

Reason: COALESCE returns the first non-NULL value.

5. Query: SELECT 'A' || NULL || 'B' FROM dual;

Output in Oracle:AB

Reason: In Oracle, NULL is treated as an empty string for character concatenation.

Q8. Difference Due to Operator Precedence

The two queries return different row counts because AND has higher precedence than OR.

Query 1

```
SELECT COUNT(*) FROM fixed_deposit
```

```
WHERE branch_id = 1 OR branch_id = 2 AND principal_amount > 500000;
```

SQL evaluates AND before OR, so this is interpreted as:

```
WHERE branch_id = 1
```

```
OR (branch_id = 2 AND principal_amount > 500000);
```

Therefore:

- **All deposits from branch 1** are included, regardless of principal amount.
- From **branch 2**, only deposits with principal_amount > 500000 are included.

Query 2

```
SELECT COUNT(*) FROM fixed_deposit
```

```
WHERE (branch_id = 1 OR branch_id = 2)
```

```
AND principal_amount > 500000;
```

The parentheses force the OR condition to be evaluated first:

```
WHERE (branch_id = 1 OR branch_id = 2)
```

AND principal_amount > 500000;

Therefore:

- Deposits from **branch 1 or branch 2** are considered.
- Only those with principal_amount > 500000 are included.

Conclusion: The first query does not apply the amount condition to branch 1, while the second query applies it to both branches. Parentheses should be used whenever the intended logical grouping is important.

Q9. Defect in the Quarterly Report

The predicate:

WHERE txn_date BETWEEN DATE '2026-01-01' AND DATE '2026-03-31'

is defective when txn_date is a TIMESTAMP or contains a time component. DATE '2026-03-31' represents midnight at the start of March 31, so transactions occurring later on March 31 are excluded.

For example:

2026-03-31 00:00:00 → included

2026-03-31 10:30:00 → excluded

2026-03-31 23:59:59 → excluded

The corrected predicate is:

WHERE txn_date >= DATE '2026-01-01'

AND txn_date < DATE '2026-04-01';

This includes every transaction from January 1 through the entire day of March 31 without depending on the time component.

Reason for the two-year under-reporting: The original predicate systematically excluded transactions occurring after midnight on the final day of every quarter.

Q10. Column Alias in ORDER BY vs WHERE

A column alias defined in the SELECT list can be used in ORDER BY because ORDER BY is logically evaluated **after** the SELECT list, so the alias already exists when sorting occurs.

However, a column alias cannot normally be used in WHERE because WHERE is logically evaluated **before** the SELECT list, so the alias has not yet been created.

Logical SQL evaluation order:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING

5. SELECT

6. ORDER BY

Example:

```
SELECT salary * 12 AS annual_salary
```

```
FROM employees
```

```
ORDER BY annual_salary;
```

Here annual_salary works in ORDER BY.

But:

```
SELECT salary * 12 AS annual_salary
```

```
FROM employees
```

```
WHERE annual_salary > 100000;
```

Normally, this is invalid because WHERE is evaluated before the annual_salary alias is created.

Instead, write:

```
SELECT salary * 12 AS annual_salary
```

```
FROM employees
```

```
WHERE salary * 12 > 100000;
```

LEVEL 2:

Q11. Create a table recurring_deposit with the given requirements.

```
CREATE TABLE recurring_deposit (
```

```
    rd_id INTEGER CONSTRAINT pk_recurring_deposit PRIMARY KEY,
```

```
    customer_id INTEGER CONSTRAINT fk_rd_customer REFERENCES customer(customer_id),
```

```
    branch_id INTEGER CONSTRAINT fk_rd_branch REFERENCES branch(branch_id),
```

```
    rd_number VARCHAR(20) CONSTRAINT uq_rd_number UNIQUE,
```

```
    monthly_amount DECIMAL(15,2) CONSTRAINT chk_rd_monthly_amount CHECK (monthly_amount  
>= 500),
```

```
    interest_rate DECIMAL(5,2) CONSTRAINT chk_rd_interest_rate CHECK (interest_rate > 0 AND  
interest_rate <= 12),
```

```
    tenure_months INTEGER CONSTRAINT chk_rd_tenure CHECK (tenure_months BETWEEN 6 AND  
120),
```

```
    start_date DATE CONSTRAINT nn_rd_start_date NOT NULL DEFAULT CURRENT_DATE,
```

```

maturity_date DATE CONSTRAINT nn_rd_maturity_date NOT NULL,

installments_paid INTEGER CONSTRAINT chk_rd_installments CHECK (installments_paid >= 0)
DEFAULT 0,

status VARCHAR(10) CONSTRAINT chk_rd_status CHECK (status IN ('ACTIVE', 'MATURED',
'DEFAULTED', 'CLOSED')) DEFAULT 'ACTIVE',

CONSTRAINT chk_rd_maturity_date CHECK (maturity_date > start_date)
);

```

Q12. Write one statement for each invalid RD and record the error message.

1. RD with monthly_amount of 100

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,
start_date, maturity_date)
```

```
VALUES
```

```
(9001, 1001, 1, 'RDINVALID001', 100, 7.10, 24, DATE '2026-03-01', DATE '2028-03-01');
```

Error:

ERROR: new row violates check constraint "chk_rd_monthly_amount"

2. RD for customer_id 5555

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,
start_date, maturity_date)
```

```
VALUES
```

```
(9002, 5555, 1, 'RDINVALID002', 5000, 7.10, 24, DATE '2026-03-01', DATE '2028-03-01');
```

Error:

ERROR: insert or update on table "recurring_deposit" violates foreign key constraint "fk_rd_customer"

3. Maturity date before start date

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,
start_date, maturity_date)
```

```
VALUES
```

```
(9003, 1001, 1, 'RDINVALID003', 5000, 7.10, 24, DATE '2026-03-01', DATE '2026-02-01');
```

Error:

ERROR: new row violates check constraint "chk_rd_maturity_date"

4. Two RDs with the same rd_number

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,  
start_date, maturity_date)
```

```
VALUES
```

```
(9004, 1001, 1, 'RD_DUPLICATE', 5000, 7.10, 24, DATE '2026-03-01', DATE '2028-03-01');
```

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,  
start_date, maturity_date)
```

```
VALUES
```

```
(9005, 1005, 5, 'RD_DUPLICATE', 2500, 6.85, 12, DATE '2026-03-05', DATE '2027-03-05');
```

Error:

ERROR: duplicate key value violates unique constraint "uq_rd_number"

5. RD with status PAUSED

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,  
start_date, maturity_date, status)
```

```
VALUES
```

```
(9006, 1001, 1, 'RDINVALID006', 5000, 7.10, 24, DATE '2026-03-01', DATE '2028-03-01', 'PAUSED');
```

Error:

ERROR: new row violates check constraint "chk_rd_status"

6. RD with installments_paid of -3

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,  
start_date, maturity_date, installments_paid)
```

```
VALUES
```

```
(9007, 1001, 1, 'RDINVALID007', 5000, 7.10, 24, DATE '2026-03-01', DATE '2028-03-01', -3);
```

Error:

ERROR: new row violates check constraint "chk_rd_installments"

Q13. Insert the three recurring deposits in a single statement.

```
INSERT INTO recurring_deposit
```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months,  
start_date, maturity_date)
```

VALUES

```
(8001, 1001, 1, 'RD2026000001', 5000, 7.10, 24, DATE '2026-03-01', DATE '2028-03-01'),  
(8002, 1005, 5, 'RD2026000002', 2500, 6.85, 12, DATE '2026-03-05', DATE '2027-03-05'),  
(8003, 1012, 4, 'RD2026000003', 10000, 7.35, 60, DATE '2026-03-10', DATE '2031-03-10');
```

Q14. Add a nullable nominee_name column and set the nominee for RD 8001.

```
ALTER TABLE recurring_deposit
```

```
ADD COLUMN nominee_name VARCHAR(60);
```

```
UPDATE recurring_deposit
```

```
SET nominee_name = 'Meera Reddy'
```

```
WHERE rd_id = 8001;
```

```
SELECT rd_id, rd_number, nominee_name
```

```
FROM recurring_deposit
```

```
WHERE rd_id = 8001;
```

Q15. Change the minimum monthly instalment to 1,000.

First, check whether any existing row violates the new rule:

```
SELECT rd_id, rd_number, monthly_amount
```

```
FROM recurring_deposit
```

```
WHERE monthly_amount < 1000;
```

Drop the existing constraint:

```
ALTER TABLE recurring_deposit
```

```
DROP CONSTRAINT chk_rd_monthly_amount;
```

Add the new constraint:

```
ALTER TABLE recurring_deposit
```

```
ADD CONSTRAINT chk_rd_monthly_amount
```

```
CHECK (monthly_amount >= 1000);
```

Q16. Add a mandatory scheme_code column to the populated table.

First attempt:

```
ALTER TABLE recurring_deposit
```

```
ADD COLUMN scheme_code VARCHAR(8) NOT NULL;
```

Error:

```
ERROR: column "scheme_code" of relation "recurring_deposit" contains null values
```

Make the operation succeed by adding the column first without NOT NULL:

```
ALTER TABLE recurring_deposit
```

```
ADD COLUMN scheme_code VARCHAR(8);
```

Populate the existing rows:

```
UPDATE recurring_deposit
```

```
SET scheme_code = 'RD2026'
```

```
WHERE scheme_code IS NULL;
```

Then make the column mandatory:

```
ALTER TABLE recurring_deposit
```

```
ALTER COLUMN scheme_code SET NOT NULL;
```

Q17. List every fixed deposit of 500,000 or more, showing receipt number, principal, rate and maturity date, largest first.

```
SELECT receipt_number,
```

```
    principal_amount AS principal,
```

```
    interest_rate AS rate,
```

```
    maturity_date
```

```
FROM fixed_deposit
```

```
WHERE principal_amount >= 500000
```

```
ORDER BY principal_amount DESC;
```

Q18. List fixed deposits booked between 1 and 10 March 2026 inclusive, sorted by booked date and then principal descending.

```
SELECT *
```

```
FROM fixed_deposit
```

```
WHERE booked_date >= DATE '2026-03-01'
```

```
    AND booked_date < DATE '2026-03-11'
```

```
ORDER BY booked_date ASC,
```

```
    principal_amount DESC;
```

Q19. Show every fixed deposit with no nominee recorded.

```
SELECT *
```

```
FROM fixed_deposit
```

```
WHERE nominee_name IS NULL;
```

Show every fixed deposit whose nominee is not 'Priya Verma', including those with no nominee.

```
SELECT *  
  
FROM fixed_deposit  
  
WHERE nominee_name <> 'Priya Verma'  
  
OR nominee_name IS NULL;
```

Explanation: A simple <> 'Priya Verma' does not include NULL values because comparisons involving NULL result in UNKNOWN, so IS NULL must also be checked.

Q20. Show annual interest and the top five fixed deposits.

```
SELECT receipt_number,  
  
       principal_amount AS principal,  
  
       ROUND(principal_amount * interest_rate / 100, 2) AS annual_interest  
  
FROM fixed_deposit  
  
ORDER BY annual_interest DESC  
  
LIMIT 5;
```

Q21. Produce a list showing receipt number, customer full name, principal and segment.

```
SELECT fd.receipt_number,  
  
       CONCAT(c.first_name, ' ', c.last_name) AS customer_full_name,  
  
       fd.principal_amount AS principal,  
  
       CASE  
  
           WHEN fd.principal_amount >= 1000000 THEN 'HIGH VALUE'  
  
           WHEN fd.principal_amount >= 100000 THEN 'STANDARD'  
  
           ELSE 'SMALL'  
  
       END AS segment  
  
FROM fixed_deposit fd  
  
JOIN customer c  
  
ON fd.customer_id = c.customer_id  
  
ORDER BY  
  
CASE  
  
    WHEN fd.principal_amount >= 1000000 THEN 1  
  
    WHEN fd.principal_amount >= 100000 THEN 2  
  
    ELSE 3  
  
END,
```

```
fd.principal_amount DESC;
```

Q22. Find UPI transactions above 50,000 and show the top ten.

```
SELECT transaction_id,  
       account_id,  
       amount,  
       narration  
FROM account_txn  
WHERE channel = 'UPI'  
       AND amount > 50000  
ORDER BY amount DESC  
LIMIT 10;
```

PostgreSQL:

```
LIMIT 10;
```

SQL Server:

```
SELECT TOP 10  
       transaction_id,  
       account_id,  
       amount,  
       narration  
FROM account_txn  
WHERE channel = 'UPI'  
       AND amount > 50000  
ORDER BY amount DESC;
```

Oracle:

```
SELECT transaction_id,  
       account_id,  
       amount,  
       narration  
FROM account_txn  
WHERE channel = 'UPI'  
       AND amount > 50000
```

ORDER BY amount DESC

FETCH FIRST 10 ROWS ONLY;

Q23. Deposit 7009 is being broken early.

Identify the deposit:

```
SELECT *
```

```
FROM fixed_deposit
```

```
WHERE deposit_id = 7009;
```

Update the status and auto-renewal:

```
UPDATE fixed_deposit
```

```
SET status = 'BROKEN',
```

```
    auto_renew = 'N'
```

```
WHERE deposit_id = 7009;
```

Verify the change:

```
SELECT deposit_id,
```

```
    status,
```

```
    auto_renew
```

```
FROM fixed_deposit
```

```
WHERE deposit_id = 7009;
```

Roll back the entire transaction:

```
ROLLBACK;
```

Q24. Nightly maturity job.

First identify the deposits:

```
SELECT *
```

```
FROM fixed_deposit
```

```
WHERE status = 'ACTIVE'
```

```
    AND maturity_date <= DATE '2026-06-30';
```

Record the count:

```
SELECT COUNT(*) AS rows_to_update
```

```
FROM fixed_deposit
```

```
WHERE status = 'ACTIVE'
```

```
    AND maturity_date <= DATE '2026-06-30';
```

Run the update:

```
UPDATE fixed_deposit  
SET status = 'MATURED'  
WHERE status = 'ACTIVE'  
AND maturity_date <= DATE '2026-06-30';
```

Confirm the count:

```
SELECT COUNT(*) AS matured_count  
FROM fixed_deposit  
WHERE status = 'MATURED'  
AND maturity_date <= DATE '2026-06-30';
```

The number of rows updated should match the count obtained before the UPDATE.

Finally, roll back:

```
ROLLBACK;
```

Q25. RD 8002 misses three instalments and is marked defaulted. Then close it instead of deleting it.

Update the status while leaving installments_paid unchanged:

```
UPDATE recurring_deposit  
SET status = 'DEFAULTED'  
WHERE rd_id = 8002;
```

Verify:

```
SELECT rd_id,  
       status,  
       installments_paid  
FROM recurring_deposit  
WHERE rd_id = 8002;
```

Close the RD without deleting the row:

```
UPDATE recurring_deposit  
SET status = 'CLOSED'  
WHERE rd_id = 8002;
```

Verify:

```
SELECT rd_id,  
       status,
```

```
installments_paid  
FROM recurring_deposit  
WHERE rd_id = 8002;
```

Explanation: The RD should be closed rather than deleted so that its transaction history, customer records, audit trail and regulatory information are preserved.

LEVEL 3:

Q26. Vendor data feed for Smart Saver

I would not store principal_amount as VARCHAR(30) in the core banking table. principal_amount is a monetary value and should be stored in an exact numeric type such as DECIMAL(15,2) or NUMERIC(15,2). I would insist that the vendor provide a clearly defined numeric contract for the field, with one consistent format such as 250000.00, and reject or quarantine invalid values such as 250000 INR if they cannot be safely interpreted. The cleaning and validation should happen in the data-ingestion/ETL pipeline before the data reaches the core database. The pipeline should remove formatting characters such as commas, validate the currency and numeric value, convert the result to DECIMAL(15,2), and send only clean, validated data to the database. The raw vendor value should be retained separately for audit and troubleshooting purposes.

Q27. ON DELETE CASCADE vs RESTRICT

I would choose RESTRICT rather than ON DELETE CASCADE for fixed_deposit.customer_id. A bank normally does not delete a customer's deposits simply because the customer relationship is closed; financial records need to be retained for audit, regulatory, tax and dispute purposes. ON DELETE CASCADE could cause deletion of deposit records automatically when a customer row is deleted, creating a risk of unintended financial-data loss. With RESTRICT, the database prevents deletion of a customer while dependent deposit records still exist, forcing the bank to follow an approved closure and retention process. The customer's account or relationship can instead be marked closed or inactive while the historical deposit records remain preserved.

Q28. Adding Employee Authorisation to account_txn

Column name: authorised_by_employee_id

Data type: INTEGER or BIGINT, matching the primary key type of the employee table.

Nullability: The column should be nullable because the requirement says the employee authorisation applies only "where applicable".

Foreign key: Yes, it should have a foreign key referencing the employee table, for example employee(employee_id), to ensure that an authorising employee actually exists.

Example:

```
ALTER TABLE account_txn  
ADD COLUMN authorised_by_employee_id INTEGER;
```

Then add the foreign key:

```
ALTER TABLE account_txn  
  
ADD CONSTRAINT fk_txn_authorised_employee  
  
FOREIGN KEY (authorised_by_employee_id)  
  
REFERENCES employee(employee_id);
```

For the 40 million existing rows, I would add the nullable column first so existing transactions remain valid, then backfill the column only where an authoritative historical source can identify the authorising employee. I would not invent employee IDs for historical transactions where the information is unavailable; those rows would remain NULL. The foreign key should be added after validating the backfilled data, using a controlled migration process to minimise impact on the large production table.

Q29. CHECK Constraint vs Lookup Table

CHECK constraint approach:

1. It is simple and easy to understand because the permitted statuses are defined directly in the fixed_deposit table.
2. It avoids an additional table and join, making queries and maintenance straightforward when the list of statuses is small and stable.

Lookup table with FOREIGN KEY:

1. It allows status values and their descriptions or additional metadata to be managed centrally.
2. It is more flexible if new statuses, display names, codes, ordering or other status-related attributes need to be added later.

For this fixed_deposit table, I would choose the CHECK constraint because there are only four clearly defined statuses and the list is small and relatively stable. A CHECK constraint provides strong database-level validation without adding unnecessary table complexity. If the bank later needs extensive status metadata or frequent changes to the allowed statuses, a lookup table would become more appropriate.

Q30. Production UPDATE Without WHERE Clause

I would immediately stop any further changes to the affected system or application process if doing so is safe. I would identify exactly when the UPDATE ran, how many rows were affected, and whether the transaction was committed. I would check the database audit logs, transaction logs and backups to determine the previous values of account.status. If the change can still be rolled back, I would immediately issue a ROLLBACK and verify the affected rows. If autocommit has already committed the change, I would use the database's recovery or point-in-time recovery facilities to restore the correct values safely. I would involve the DBA and relevant incident-response team before performing a production restoration. I would verify the repaired data using counts, samples and reconciliation with trusted records. I would document the incident and its impact without blaming the individual. First working-practice change: require code review and approval for production data-changing statements.

Second working-practice change: require a SELECT preview, transaction-based execution and mandatory WHERE-condition checks before production UPDATE or DELETE operations.

